

Operations Manual

One truck, one driver, one branch — followed from an empty organization to a cost-per-kilometre figure you can defend.

Worked example	AFU0078 · a truck
Driver	Tendai Moyo
Branch	Harare
Companion docs	ROLE_RESPONSIBILITY_MATRIX · DATA_FLOW_EXAMPLES · DATA_ENTRY_GUIDE

Operations Manual

One truck, one driver, one branch, followed from an empty organization to a cost-per-kilometre figure you can defend in a board meeting.

Everything in this manual is a real screen, a real field and a real endpoint in this build. Where the platform cannot yet do something, it says so rather than describing an intention.

The worked example, used throughout:

Branch	Harare
Vehicle	AFU0078 — a truck
Driver	Tendai Moyo

Before you start: the one rule that explains the order

Every operational record inherits its org unit from the vehicle it names. Fuel, expenses, trips, maintenance and work orders all do. A record with no org unit is visible only to organization-wide roles.

Two consequences, and they are the reason the order below is not a suggestion:

- **Branches before vehicles.** A vehicle created before its branch exists has no unit to inherit, and every cost recorded against it is invisible to the branch manager who is accountable for it.
- **Vehicles before everything else.** Nothing that references a vehicle can be recorded first.

Skipping a step does not produce an error. It produces a module that looks empty.

STEP A — Create the Harare branch

Where Administration → Organization → Teams (/organizations/teams)

Who `organization_admin` or `organization_owner` (`org_unit:manage`). A `branch_manager` can **view** the tree but not change it.

FIELD	FOR AFU0078'S BRANCH	WHY IT MATTERS
Type	<code>branch</code>	The hierarchy is enforced: Branch → Department → Workshop → Fleet. A department cannot be created at the top level.
Name	<code>Harare</code>	What appears on every scoped screen.
Code	<code>HRE</code>	Optional; useful in exports.
Parent	<i>(none — top level)</i>	A branch sits at the root.
Manager	Tendai's manager, if known	Informational; it does not grant access.

What happens afterwards. `tblorgunits` gets a row with a materialised `path[]` and `depth`, which is what makes "this branch and everything under it" a single indexed query rather than a recursive walk. An `OrgUnitCreated` event fires.

Assigning a manager here does not give them access. Access comes from a scope assignment (Step B). The two are separate on purpose: the person accountable for a branch and the people who may see its data are different lists.

Platform administrators: `/api/platform/organizations/:id/org-units` now exists, so a super admin can build another customer's branch tree from the platform-admin screens. This was impossible until this round — the tenant-scoped endpoints resolve the organization from the caller's own session, so pointing that UI at them would have shown an admin their *own* branches under someone else's name.

STEP B — Create the user who will run Harare

Where Administration → Members (`/organizations/members`), then Administration → Scope assignments

Who `organization_admin` (`user:create` , `scope_assignment:manage`)

Two things, and both are required:

1. **The account.** Invite by email; the invitation creates the account inside this organization.
2. **The scope assignment.** User → org unit → role. For a Harare branch manager: user, `Harare` , `branch_manager` .

A user with no scope assignment sees nothing. That is fail-closed by design, not a bug: `accessibleOrgUnitIds` resolves to `[]` , which becomes `{orgUnitId: {$in: []}}` on every list and matches nothing. It is the first thing to check when someone reports missing data.

What `branch_manager` can then do (from `ROLE_RESPONSIBILITY_MATRIX.md` , generated from the permission table):

- create and edit vehicles; create, edit and delete trips, fuel, expenses and maintenance — **within Harare**;
- assign drivers, run dispatch and the shift roster, approve expenses and bookings;
- **read** finance, and not post to it. A branch manager is accountable for their branch's cost per km and must see it; posting depreciation and submitting GL figures are accounting functions.

STEP C — Create the vehicle AFU0078

Where Vehicles → New vehicle (`/vehicles`)

Who `vehicle:create` — `branch_manager` , `fleet_manager` , `organization_admin`

FIELD	VALUE	NOTES
<code>license_plate</code>	<code>AFU0078</code>	Stored uppercase. It is the key every operational record joins on, it is mutable, and nothing in the schema enforces uniqueness — see the warning below.
<code>make</code> / <code>model</code> / <code>year</code>	e.g. <code>DAF</code> / <code>CF 85</code> / <code>2019</code>	<code>year</code> feeds average fleet age. A vehicle with no year is now excluded from that average rather than dated to a default.
<code>vehicle_type</code>	<code>Truck</code>	Free text. The live map resolves it to a silhouette tolerantly — real data in this deployment contains <code>" DAF "</code> .
<code>fuel_type</code>	<code>Diesel</code>	Pre-fills the fuel form when this vehicle is selected.
<code>odometer</code>	<code>current reading</code>	The starting point for distance-by-odometer trips.
<code>service_interval</code>	e.g. <code>15000</code>	Drives service-due calculations.
<code>registration_expiry</code>	<code>date</code>	Drives the expiry badges on the vehicle's page.

FIELD	VALUE	NOTES
vin	17 characters	Optional, validated for length.

The vehicle inherits **the creating user's** org unit — a vehicle has no other vehicle to inherit from. Create it as the Harare branch manager, or as an admin who sets Harare explicitly.

Two vehicles must never share a plate. This deployment has two distinct organizations both named "Toyota Zimbabwe", and plates carry no unique index. A duplicate plate is now **refused with 409** at every write path rather than silently resolving to whichever row Mongo returned first — but resolve the duplicate in Vehicles, because until you do, nothing can be recorded against either vehicle.

STEP D — Create the driver Tendai Moyo

Where Drivers → New driver (/drivers)

Who gated on VEHICLE_CREATE / VEHICLE_EDIT today (see the note below)

FIELD	VALUE	NOTES
name	Tendai Moyo	
driver_code	badge or staff number	An alternative lookup key for spreadsheet import.
email / phone		
license_number		
license_expiry	date	Feeds compliance.
status	active	Historic rows often carry no status at all, so "active" is read as <i>explicitly active OR field absent</i> .

A driver has no vehicle to inherit from, so the driver takes **the submitter's** org unit. Create Tendai as the Harare branch manager and he belongs to Harare.

This was the "POST 201, then GET empty" bug. The org unit was computed in the controller and then dropped twice — once by a zod schema that strips undeclared keys, once by an allow-list payload that never named the field — while the scoped list filtered on it. The record saved and vanished. Deriving the value inside the same function that builds the payload is what fixed it.

There is no Permission.DRIVER_VIEW or DRIVER_CREATE. Driver management is gated on the vehicle permissions as a stopgap; only `driver:assign` and `driver:view:trips` exist in their own right. The navigation mirrors that exactly rather than inventing a permission the API does not enforce.

STEP E — Assign Tendai to AFU0078

Where Vehicles → AFU0078 → Driver tab

Who `driver:assign`

The panel shows the current assignment and lets you change or clear it. Both directions publish an event (`VehicleDriverAssigned` / `VehicleDriverUnassigned`).

What this assignment is, and is not:

- It is the vehicle's *current* driver — used by dispatch, by the driver's scorecard, and shown on the vehicle's page.
- It is **not** applied backwards. A fuel log records who fuelled the vehicle *on that date*; a trip records who drove it *on that trip*. Back-filling the present driver onto historical records would move one person's fuel spend and risk score onto another, and would change again at every reassignment.

A driver **outside your branch** is refused, and the refusal is worded identically to "no such driver" so it cannot be used to enumerate another branch's roster. A driver with **no** org unit is assignable only by an organization-wide role — they do not appear in a branch manager's driver list, so they must not be assignable from it either.

STEP F — Record a fuel log

Where Vehicles → AFU0078 → **Log fuel** (the action bar under the title), or Fuel → New entry

Who `fuel:create` — including `driver`, so Tendai can log his own refuel

Opening it from the vehicle's page pre-selects AFU0078 and titles the dialog "*Log fuel for AFU0078*". The picker stays editable so a wrong turn is correctable in place.

FIELD	VALUE	NOTES
<code>license_plate</code>	AFU0078	Pre-filled.
<code>date</code>	the refuel date	The record's own date is the accounting period. A log entered today for last month's refuel posts to last month.
<code>fuel_volume</code>	e.g. 180	Litres, by the unit selected.
<code>unit_id</code>	L	
<code>cost</code>	e.g. 245.00	
<code>currency</code>	USD	Absent means the tenant's reporting currency. A foreign currency with no FX rate is refused , never converted at 1:1.
<code>odometer</code>	the reading at the pump	Enter this. <code>0</code> means "not recorded", and without it fuel efficiency for this vehicle cannot be computed.
<code>driver_id</code>	Tendai Moyo	See below.
<code>payment_method</code>	cash / fuel card / ...	A fuel-card payment requires a card, and the card must be active.
<code>fuel_station_id</code>	optional	
<code>is_full_tank</code>	tick when it was	Improves efficiency calculations.

The driver field works for the first time in this build. The field was never declared on the validation schema, and a plain `z.object` strips undeclared keys — so the value the form sent, the value the spreadsheet importer resolved from a driver's *name*, and the value the handler copied into its payload were all deleted in transit. Every fuel log ever written was unattributed, which is why *Fuel Cost by Driver* showed a single bar labelled **Unassigned**.

An attribution can now also be **removed**: choosing "Unassigned" on an edit clears it. Previously a log pointed at the wrong driver could be re-pointed but never returned to unattributed.

Where this one record now appears

SCREEN	WHAT IT SHOWS
Fuel → Logs	the entry
Vehicles → AFU0078 → Activity	"Fuel logged · 180 L · USD 245.00"
Fuel → Analytics → Cost by Driver	Tendai Moyo , not Unassigned
Allocation ledger	a posting, <code>costCategory: fuel</code> , dated to the refuel
Cost per km	the cost half of the ratio
Digital twin	the vehicle's fuel state
Fuel-fraud detection	a data point for that vehicle's baseline

STEP G — Record an expense

Where Vehicles → AFU0078 → **Add expense**

Who `expense:create`

FIELD	VALUE	NOTES
<code>license_plate</code>	AFU0078	Pre-filled.
<code>amount</code>	e.g. 60.00	
<code>date</code>	the date incurred	Again, the accounting period.
<code>expense_type_id</code>	e.g. Tollgate	<code>tblexpense_types</code> is a global catalogue with no tenantId — re-seeding it changes every category id on every existing expense.
<code>currency</code>	USD	Same FX rule as fuel.
<code>description</code>	free text	This is what the Activity timeline shows.

Financial impact. An `ExpenseCreated` event posts the amount into `tblallocationledger` under `costCategory: 'expense'`, keyed by a deterministic idempotency key so a redelivered event writes nothing. It joins fuel in the numerator of cost per km.

STEP H — Schedule a service

Where Vehicles → AFU0078 → **Schedule service**

Who `maintenance:create`

FIELD	VALUE	NOTES
<code>license_plate</code>	AFU0078	Pre-filled.
<code>title</code>	"60,000 km service"	
<code>due_date</code>	date	

FIELD	VALUE	NOTES
priority	medium	
service_type / category		
estimated_cost	if you have one	Leave blank if you do not. The forecast now shows "No estimate" rather than inventing \$500 per line, which used to be summed into a figure managers budgeted against.
recurrence_interval	e.g. every 15,000 km	Completing a recurring reminder creates the next one.

What the vehicle joins. It appears in Maintenance → Upcoming, in the maintenance calendar, on the vehicle's Activity timeline, and — once due — in the Command Centre's attention feed.

On completion, a `ReminderCompleted` event posts `estimated_cost` into the ledger as `costCategory: 'maintenance'`, labelled "*estimated cost — no actuals recorded*". Reminders carry no actuals field, and saying so on the posting stops a finance user reconciling an estimate against an invoice and concluding the ledger is wrong.

STEP I — Raise a work order

Where Vehicles → AFU0078 → **Raise work order**, or Work orders → New work order

Who `workorder:create` — `fleet_manager`, `workshop_manager`, `organization_admin`. A `branch_manager` can **assign and manage** work orders but not create them.

FIELD	VALUE
Vehicle	AFU0078 (pre-filled)
What needs doing	"Replace nearside mirror"
Priority	low / medium / high / critical
Details	anything the mechanic needs before starting

This is a new flow in this build. Work orders previously arrived only from a DVIR defect, a maintenance reminder, or a dispatched attention item — `POST /api/workorders` had shipped, permission-gated and tested, reachable by other code and by nobody in a workshop.

The lifecycle, and where the money enters

open → assigned → in progress → completed

- **Assign** (`workorder:assign`) — a mechanic and optionally a bay. Stops the SLA response clock.
- **Parts** — consumed stock, written to `tblstockmovements`.
- **Labour** — hours and cost.
- **Complete** (`workorder:complete`) — `WorkOrderCompleted` fires.

A completed work order posts TWO ledger lines, not one: parts under `maintenance`, labour under `other`. They are different costs to a finance team, and `totalCost` alone makes the split unrecoverable. The total is used only when neither component is present — posting both would double-count, and on an append-only ledger that needs a human reversal.

This branch was unreachable until this build. The event was in the posting handler's map but missing from the subscription list, under a comment claiming the handler saw every event. Every part and every labour hour on every completed work order stayed out of the ledger. The subscription is now derived from the map itself.

STEP J — A trip

There are two kinds, and the difference matters more than it looks.

Generated from telemetry (preferred)

If AFU0078 has a tracker and the provider integration is enabled, trips appear on their own. Every 10 minutes a scheduled sweep reads new readings from a per-vehicle watermark and detects journeys:

- ignition is authoritative when the device reports it;
- otherwise movement above 5 km/h counts as driving;
- a stop of 5 minutes ends a trip;
- **silence of more than 60 minutes** ends it at the last known fix;
- journeys under 2 minutes *and* 0.5 km are discarded as yard shuffling.

These trips carry `distance_km_known`, a start and end position, a maximum speed, and — this is the point — **a replayable route**.

If your trackers report less often than hourly, lower `signalGapMinutes` deliberately. At the original default of 30 minutes the rule fired on ordinary operation: every journey split into single-fix stubs, each failed the minimum-trip test and was discarded, and **a full day of driving produced zero trips**. Silence, not a wrong answer.

Entered by hand

Where Vehicles → AFU0078 → **Log trip**

Who `trip:create` — including `driver` and `dispatcher`

FIELD	NOTES
<code>mode</code>	<code>distance</code> (you know the kilometres) or <code>odometer</code> (start and end readings)
<code>date</code>	
<code>unit_id</code>	a distance unit; the form only offers those
<code>driver_id</code>	Tendai. Scope-checked, like the vehicle.
<code>start_time</code> / <code>end_time</code>	optional, but duration and average speed are derived from them and cannot be computed without both

A manual trip has no positions, so **playback will say so** rather than showing an empty map: *"This trip has no start and end time"*.

Playback

Open any generated trip (Trips → the trip) and the **Route playback** panel replays it: the route line, start and end markers, a scrubber, play/pause, a speed control, and a readout of time, speed and heading at the playhead.

Two things it deliberately will not do:

- **It never extrapolates.** The playhead interpolates *between* two received fixes and stops at the last one. Drawing beyond it would put the vehicle somewhere the platform has no evidence it has been, rendered identically to a measured position.
- **It never blends speed.** Speed and heading are shown as reported and read "Not reported" when the reading did not carry them — 0 km/h means stopped and a heading of 0 means due north.

A long track is evenly sampled, always keeping the first and last point, and the panel says so when it has been thinned.

STEP K — Follow the data

One truck, one driver, five records. Here is everywhere they now appear.

AFU0078's own page

TAB	WHAT IT SHOWS
Overview	make, model, odometer, registration, service interval — and Recent activity , the six most recent things that happened to this vehicle
Specifications	the full record
Driver	Tendai, with the assignment history
Analytics	fuel, expense, trip and maintenance analytics for this vehicle alone
Costs	the allocation ledger for this vehicle: fuel, maintenance, expenses, depreciation
Activity	the full merged history — fuel, expenses, trips, services, work orders, and record changes, newest first

The action bar under the title carries exactly the actions **you** hold the permission for. A viewer sees none of it; a workshop manager sees "Schedule service" and "Raise work order" and no fuel button, because `fuel:create` is not theirs.

Across the platform

WHERE	AFU0078 APPEARS AS
Dashboard	one of the fleet KPIs; in the maintenance-due count if the service is near
Fuel → Analytics	a bar in Cost by Vehicle, and Tendai a bar in Cost by Driver
Expenses → Analytics	in the cost breakdown by vehicle
Trips → Analytics	in vehicle utilisation, and Tendai in driver utilisation
Maintenance → Upcoming	until the service is completed
Work orders	the mirror job, until it is completed
Command Centre	if a rule or model raises something about it — with the evidence that produced it
Reports → Cost & Utilization	inside cost per km for Harare
Allocation ledger	one posting per cost, each traceable back to its source record
Value ledger	only if someone resolved an attention item about it with a stated outcome
ESG export	in the fleet-health section

The chain, in one line

```
fuel + expenses + maintenance + work orders → allocation ledger → cost per km →  
Harare's number  
telemetry → trips (distance_km_known = true)
```

Miss any one input and the figure is either `null` or understated. It is never quietly wrong: cost per km is **`null` at zero distance, never 0**, because "we cannot compute this" and "this costs nothing to run" are different claims.

What to check when a screen looks empty

SYMPTOM	CHECK
A record saved and then vanished	Org-unit assignment. <code>POST 201</code> , <code>GET empty</code> is the signature.
A branch manager sees less than an admin	That is correct. Never validate a scoping fix from an org-wide account — it applies no filter and proves nothing.
Cost per km is <code>null</code>	No trip distance in the period, or every trip has <code>distance_km_known: false</code> .
Cost per km looks too low	Records predating the ledger fixes. Run <code>npm run finance:backfill-ledger</code> (dry run first).
"Unassigned" in Cost by Driver	Fuel logs written before this build carry no driver, permanently. New ones do.
No trips at all	<code>tbltrip_detection_state</code> — if <code>tbltrips</code> was cleared without it, every watermark sits in the future and the sweep skips everything.
A figure reads 0	Ask whether it should read "Not measured". Several such defaults were removed this round.

The order, one more time

```
1 Branch          ← everything below inherits from this  
2 User + scope    ← no assignment means no visibility  
3 Vehicle         ← nothing referencing a vehicle can come first  
4 Driver  
5 Assignment  
6 Fuel · Expense · Maintenance · Work order · Trip  
7 Analytics, cost per km, attention items — these compute themselves
```

Steps 1–5 are configuration and are done once. Step 6 is the operation. Step 7 is what the customer bought.